Sugen ICD-11 Coder Rebuild and Verification Plan

> **For Hermes:** Use this plan task-by-task with a fresh implementation context, TDD for every code change, and a review after each coherent milestone. Do not delete or modify supplied company files.

Goal: Build an honest, maintainable, no-key ICD-11 MMS 2026-01 coding system that satisfies Sugen's exact Docker command, avoids development-set leakage, and is verified against the supplied scorer before any submission.

Architecture: Keep the supplied `public/` tree unchanged and place the implementation at the workspace root using `app/`, `tests/`, `Dockerfile`, and `docker-compose.yml`. Use a deterministic offline-first pipeline: parse JSONL, normalize the note, retrieve and rank valid MMS catalog candidates, emit up to three codes incrementally, and optionally add a separately tested API reranker only if it improves measured validation without becoming required. Development gold labels may be used by tests and analysis, never as runtime features for the submitted coder.

Tech Stack: Python 3.11+ standard library first; Docker Compose; pytest for tests; supplied `public/score.py`; optional HTTP API adapter only behind an explicit environment variable; Git for real history.

---

1. Current context and assumptions

Workspace boundary

The only project workspace is:

```
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health
```

All project files, plans, code, tests, reports, outputs, and feedback must remain under that path. Do not write project artifacts to another Desktop/work folder.

Supplied files to preserve

Never delete, overwrite, move, or reformat these during implementation:

```
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\SugenEngTest.tgz
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\public\BRIEF.md
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\public\score.py
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\public\data\dev_notes.jsonl
```

```
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\public\data\mms-2026-01.json
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\public\data\predictions.example
```

The extracted data has been inspected: 60 development notes, 18,375 MMS catalog rows, and the official scorer are present. The exact email timestamp is not available in the pasted email, so the deadline remains “within three days of the sending timestamp” until the original email metadata is supplied.

Client requirements to preserve as acceptance criteria

- Exact run command: `docker compose run coder /in/notes.jsonl /out/predictions.jsonl`.
- One output JSON object per non-empty input line.
- Every input ID must appear exactly once in output.
- Output codes are at most three, best first, and must exist in MMS 2026-01.
- No API key is required for a complete run.
- The grader runs 600 hidden notes within 60 minutes on 8 CPUs, 16 GB RAM, and no GPU.
- The solution may use OpenAI or Anthropic APIs only through the allowed runtime network/proxy, but any API path must have a robust no-key fallback.
- Predictions must be flushed as the run progresses.
- Deliverables are a Git bundle with real history, a one-page README, `DECISIONS.md`, and an honest hours-spent line.
- The final repo must be tested from a clean clone before submission.

Quality and honesty rules

- Do not use `gold` fields from `dev_notes.jsonl` in the runtime coder.
- Do not fabricate commits, timings, experiments, scores, or hidden-score estimates.
- Do not intentionally add mistakes to look human; use clear naming, modest abstraction, rationale comments, focused tests, and real history.
- Distinguish measured development performance from unknown hidden performance.

---

2. Proposed repository layout

Create only these project paths under the canonical root, leaving `public/` unchanged:

```
sugen_health/
├── .git/
├── .gitignore
├── .hermes/plans/2026-11-04_122300-sugen-icd11-rebuild.md
├── app/
│   ├── __init__.py
│   ├── __main__.py
│   ├── catalog.py
│   ├── normalize.py
```

```
│   ├── ranking.py
│   ├── io.py
│   ├── coder.py
│   └── settings.py
├── tests/
│   ├── conftest.py
│   ├── test_catalog.py
│   ├── test_normalize.py
│   ├── test_ranking.py
│   ├── test_io.py
│   ├── test_coder.py
│   └── fixtures/
├── analysis/
│   ├── gap-analysis.md
│   └── baseline-results/
├── outputs/
├── feedback/
├── Dockerfile
├── docker-compose.yml
├── README.md
└── DECISIONS.md
```

`public/` remains at the root because the supplied relative paths and scorer must remain valid. The original `SugenEngTest.tgz` also remains untouched.

---

3. Step-by-step implementation tasks

Each code task follows the same TDD cycle: write the failing test, run it and record the failure, implement the smallest change, run the focused test and full suite, then commit the real change.

Task 1: Record the baseline and initialize real history

Objective: Establish a reproducible starting point without touching supplied files.

Files:

- Create: `.gitignore`
- Create: `analysis/gap-analysis.md`
- Create: `DECISIONS.md`
- Create: `README.md`
- Do not modify: `SugenEngTest.tgz`, `public/`

Steps:

1. Record the archive and supplied-file SHA-256 hashes in `analysis/gap-analysis.md`.
2. Create `.gitignore` with:

```
__pycache__/
*.py[cod]
.pytest_cache/
.venv/
outputs/*
!outputs/.gitkeep
analysis/baseline-results/*
!analysis/baseline-results/.gitkeep
```

3. Create `DECISIONS.md` with the initial date/time, the no-leakage rule, the offline-first choice, and alternatives considered.
4. Create a README stub that clearly says implementation is in progress and contains no unsupported score.
5. Run:

```
cd /c/Users/rajiv/OneDrive/Desktop/work/sugen_health
git init -b main
git status --short
```

Expected: `public/` and `SugenEngTest.tgz` are visible as untracked or ignored only after deliberate rule decisions; no files are deleted or altered.

6. Commit only the baseline metadata:

```
git add .gitignore README.md DECISIONS.md analysis/gap-analysis.md
git commit -m "chore: establish assessment workspace and constraints"
```

Task 2: Add the Python test harness and smoke test

Objective: Make the test command work before implementing domain code.

Files:

- Create: `app/__init__.py`
- Create: `tests/conftest.py`
- Create: `tests/test_smoke.py`

Failing test:

```
from app import __version__
```

```
def test_application_package_has_version():
    assert __version__ == "0.1.0"
```

Run:

```
python -m pytest tests/test_smoke.py -q
```

Expected: FAIL because `app.__version__` does not exist.

Implement `app/__init__.py`:

```
"""Offline-first ICD-11 coding application."""

__version__ = "0.1.0"
```

Run the same command. Expected: `1 passed`.

Commit:

```
git add app tests
 git commit -m "test: establish Python package smoke test"
```

Task 3: Define catalog loading and code validation

Objective: Load only the supplied MMS catalog and expose normalized valid-code lookup.

Files:

- Create: `app/catalog.py`
- Create: `tests/test_catalog.py`

Failing tests:

```
from pathlib import Path
from app.catalog import load_catalog

CATALOG = Path("public/data/mms-2026-01.json")


def test_catalog_loads_all_supplied_rows():
    catalog = load_catalog(CATALOG)
    assert len(catalog) == 18_375
    assert catalog[0].code == "1A00"
```

```
def test_catalog_rejects_unknown_code():
    catalog = load_catalog(CATALOG)
    assert catalog.is_valid("1A00")
    assert not catalog.is_valid("NOT-A-CODE")
```

Run:

```
python -m pytest tests/test_catalog.py -q
```

Expected: FAIL because `load_catalog` and `Catalog` do not exist.

Implement a small immutable record and catalog loader in `app/catalog.py`. Requirements:

- parse JSON once at startup;
- preserve `code`, `title`, `chapter`, `leaf`, `residual`, `depth`, and `uri`;
- normalize lookup keys to uppercase and strip postcoordination stems only where the scorer does;
- reject malformed catalog rows with a clear exception;
- never load `gold` labels here.

Run the focused tests and then:

```
python -m pytest tests/test_catalog.py -q
```

Expected: `2 passed`.

Commit:

```
git add app/catalog.py tests/test_catalog.py
git commit -m "feat: load and validate ICD-11 MMS catalog"
```

Task 4: Implement deterministic text normalization

Objective: Normalize consultation text without discarding clinically meaningful negation or numeric detail.

Files:

- Create: `app/normalize.py`
- Create: `tests/test_normalize.py`

Failing tests:

```
from app.normalize import normalize_text, tokenize


def test_normalization_is_case_and_whitespace_stable():
    assert normalize_text("  Fever\\nWITH chills. ") == "fever with chills."


def test_tokenizer_keeps_negation_terms():
    assert "no" in tokenize("No cough or fever")
    assert "cough" in tokenize("No cough or fever")
```

Run and verify failure:

```
python -m pytest tests/test_normalize.py -q
```

Implement only reusable normalization/tokenization functions. Preserve words such as `no`, `not`, `without`, `negative`, `excluded`, and `denies`; do not silently remove them as generic stop words.

Run:

```
python -m pytest tests/test_normalize.py -q
```

Expected: `2 passed`.

Commit:

```
git add app/normalize.py tests/test_normalize.py
git commit -m "feat: normalize clinical note text"
```

Task 5: Build catalog-only candidate retrieval

Objective: Retrieve plausible valid candidates from titles and chapter metadata without using development gold labels.

Files:

- Create: `app/ranking.py`
- Create: `tests/test_ranking.py`

Failing tests:

```python
from pathlib import Path
from app.catalog import load_catalog
from app.ranking import rank_candidates

CATALOG = load_catalog(Path("public/data/mms-2026-01.json"))


def test_malaria_note_ranks_malaria_code_first():
    ranked = rank_candidates(
        "Assessment: uncomplicated falciparum malaria with fever and chills.",
        CATALOG,
        limit=5,
    )
    assert ranked[0].code == "1F40.Z"


def test_candidates_are_catalog_codes_only():
    ranked = rank_candidates("dry cough and fever", CATALOG, limit=5)
    assert 1 <= len(ranked) <= 5
    assert all(CATALOG.is_valid(row.code) for row in ranked)
```

Run and confirm failure:

```
python -m pytest tests/test_ranking.py -q
```

Implement a simple, explainable ranking pipeline:

1. tokenize the note;
2. score title-token overlap with inverse document frequency from the catalog only;
3. add exact normalized phrase bonuses for title matches;
4. add cautious diagnosis/impression line bonuses;
5. prefer leaf codes when scores are otherwise close;
6. penalize candidates contradicted by explicit negation only where the phrase match is unambiguous;
7. return a deterministic tie-break order by score, specificity/depth, then code.

Do not use `public/data/dev_notes.jsonl` inside `app/`.

Run:

```
python -m pytest tests/test_ranking.py -q
```

Expected: `2 passed`. If the first test reveals that catalog titles alone cannot support the expected code, record the failure in `DECISIONS.md` and revise the documented rule set rather than injecting gold labels.

Commit:

```
git add app/ranking.py tests/test_ranking.py DECISIONS.md
git commit -m "feat: rank catalog-only ICD-11 candidates"
```

Task 6: Add conservative specificity and fallback behavior

Objective: Ensure the first result is the most specific supported candidate without inventing note details.

Files:

- Modify: `app/ranking.py`
- Modify: `tests/test_ranking.py`

Failing tests: Add tests for:

```
def test_unspecified_code_is_allowed_when_detail_is_absent():
    ranked = rank_candidates("confirmed malaria, species not documented",
CATALOG, limit=3)
    assert ranked[0].code in {"1F40.Z", "1F40"}


def test_parent_does_not_beat_supported_leaf_on_equal_evidence():
    ranked = rank_candidates("moderate asthma attack", CATALOG, limit=10)
    assert ranked[0].leaf is True
```

Run the focused tests and verify failure before implementation.

Implement only evidence-based specificity rules:

- use explicit species/severity/site/type details when present;
- use residual/unspecified codes when those details are absent;
- never infer age, sex, severity, laterality, or etiology solely from a common association;
- keep parent candidates as fallbacks only when no supported child outranks them.

Run:

```
python -m pytest tests/test_ranking.py -q
```

Expected: all ranking tests pass. Commit:

```
git add app/ranking.py tests/test_ranking.py
git commit -m "feat: apply evidence-based code specificity"
```

Task 7: Implement streaming JSONL input and output

Objective: Read every input note and flush one valid prediction line immediately.

Files:

- Create: `app/io.py`
- Create: `tests/test_io.py`

Failing tests:

```
import json
from app.io import iter_notes, write_prediction


def test_iter_notes_skips_blank_lines_but_preserves_ids(tmp_path):
    path = tmp_path / "notes.jsonl"

path.write_text('{"id":"a","text":"fever"}\\n\\n{"id":"b","text":"cough"}\\n',
encoding="utf-8")
    assert [row["id"] for row in iter_notes(path)] == ["a", "b"]


def test_write_prediction_limits_codes_to_three(tmp_path):
    path = tmp_path / "pred.jsonl"
    write_prediction(path, "a", ["1A00", "1A01", "1A02", "1A03"])
    row = json.loads(path.read_text(encoding="utf-8"))
    assert row == {"id": "a", "codes": ["1A00", "1A01", "1A02"]}
```

Run and verify failure, then implement:

- strict JSON decoding with line-numbered errors;
- required string `id` validation;
- missing `text` treated as empty text rather than crashing the whole run;
- output parent directory creation;
- append/write behavior that flushes after each line;
- maximum three code enforcement.

Run:

```
python -m pytest tests/test_io.py -q
```

Expected: `2 passed`.

Commit:

```
git add app/io.py tests/test_io.py
git commit -m "feat: stream JSONL notes and predictions"
```

Task 8: Wire the no-key coder entry point

Objective: Make the deterministic coder executable with exactly two positional paths.

Files:

- Create: `app/settings.py`
- Create: `app/coder.py`
- Create: `app/__main__.py`
- Create: `tests/test_coder.py`

Failing integration test:

```
import json
import subprocess
import sys


def test_coder_generates_one_prediction_per_note(tmp_path):
    source = tmp_path / "notes.jsonl"
    output = tmp_path / "out" / "predictions.jsonl"
    source.write_text(
        json.dumps({"id": "n1", "text": "Assessment: uncomplicated malaria"}) +
"\\n",
        encoding="utf-8",
    )
    result = subprocess.run(
        [sys.executable, "-m", "app", str(source), str(output)],
        capture_output=True,
        text=True,
    )
    assert result.returncode == 0, result.stderr
    rows = [json.loads(line) for line in output.read_text().splitlines()]
    assert len(rows) == 1
    assert rows[0]["id"] == "n1"
    assert 1 <= len(rows[0]["codes"]) <= 3
```

Run:

```
python -m pytest tests/test_coder.py -q
```

Expected: FAIL because the entry point does not exist.

Implement `app/coder.py` so startup loads only:

```
app/../public/data/mms-2026-01.json
```

The main loop must:

1. parse arguments as `INPUT_JSONL OUTPUT_JSONL`;
2. load catalog once;
3. iterate input rows;
4. rank candidates from catalog-only data;
5. filter to valid codes;
6. write and flush one output line;
7. continue only when a recoverable per-note error can be represented safely.

Implement `app/__main__.py` as:

```
from .coder import main

if __name__ == "__main__":
    main()
```

Run:

```
python -m pytest tests/test_coder.py -q
python -m app public/data/dev_notes.jsonl outputs/dev_predictions.jsonl
python public/score.py --gold public/data/dev_notes.jsonl --pred
outputs/dev_predictions.jsonl --mms public/data/mms-2026-01.json
```

Expected: integration tests pass; the command emits 60 prediction lines; scorer reports a real measured development score. Do not optimize toward 75/75 by using gold labels.

Commit:

```
git add app tests
 git commit -m "feat: add deterministic ICD-11 coder entry point"
```

Task 9: Add explicit output validation tests

Objective: Catch scorer-invalid outputs before Docker testing.

Files:

- Create: `tests/test_output_contract.py`
- Modify: `app/io.py` or `app/coder.py` only if tests identify a defect.

Add tests that assert:

- a code not in the catalog is never emitted;
- a fourth candidate is dropped;
- duplicate input IDs are reported or handled according to a documented policy;
- empty input produces an empty output without crashing;
- every normal input row produces exactly one output row;
- output IDs preserve input IDs exactly.

Run:

```
python -m pytest tests/test_output_contract.py -q
python -m pytest -q
```

Expected: all tests pass. Record any behavior choice in `DECISIONS.md` and commit:

```
git add app tests DECISIONS.md
git commit -m "test: enforce prediction output contract"
```

Task 10: Add optional API reranking only if justified

Objective: Keep the no-key path complete while allowing a measured optional API path.

Files:

- Create: `app/llm.py` only if baseline analysis justifies it.
- Create: `tests/test_llm.py` with mocked HTTP responses.
- Modify: `app/settings.py`, `app/coder.py` only after tests exist.

Do not start with network calls. First run the deterministic baseline and inspect errors. If an API path is justified:

1. write a failing mocked test for valid JSON selection from a bounded candidate list;
2. write a failing mocked test for timeout/malformed response fallback;
3. implement the smallest client using `OPENAI_API_KEY` or `ANTHROPIC_API_KEY` only when explicitly configured;
4. constrain responses to retrieved valid candidates;
5. never allow API failure to produce a missing prediction;
6. never send the whole 18,375-code catalog in one prompt;
7. document model, cost, proxy, timeout, retry, and fallback decisions.

Run:

```
python -m pytest tests/test_llm.py -q
python -m pytest -q
```

Expected: mocked tests pass without network access. Commit only if the measured development error analysis supports the added complexity:

```
git add app tests DECISIONS.md
 git commit -m "feat: add optional constrained API reranking"
```

Task 11: Create the Docker image and Compose service

Objective: Make the required command work in a clean build with no API key.

Files:

- Create: `Dockerfile`
- Create: `docker-compose.yml`
- Create: `.dockerignore`
- Create: `tests/test_container_contract.py` if container tests are feasible locally.

`Dockerfile` requirements:

- use a pinned or clearly documented Python 3.11 slim base;
- copy only application code and supplied MMS data needed at runtime;
- set a non-interactive entrypoint;
- do not require network access at run time for the no-key path.

`docker-compose.yml` requirements:

- define service `coder`;
- mount the root input path read-only at `/in`;
- mount `./outputs` at `/out`;
- pass optional API environment variables without requiring them;
- preserve the exact command override behavior.

Run:

```
cd /c/Users/rajiv/OneDrive/Desktop/work/sugen_health
docker compose config --quiet
docker compose build --no-cache coder
```

Expected: config exits 0 and image build exits 0.

Run the exact required command using a disposable input copy under the workspace:

```
cp public/data/dev_notes.jsonl notes.jsonl
docker compose run --rm coder /in/notes.jsonl /out/predictions.jsonl
```

Expected: container exits 0 and `outputs/predictions.jsonl` contains 60 lines. If the exact brief command requires root-level `out/`, create that path only after documenting the path choice; do not silently alter the supplied `public/` tree.

Run scorer:

```
python public/score.py --gold public/data/dev_notes.jsonl --pred
outputs/predictions.jsonl --mms public/data/mms-2026-01.json
```

Expected: zero missing predictions and zero invalid codes; record the measured score. Commit:

```
git add Dockerfile docker-compose.yml .dockerignore
 git commit -m "build: package coder for required Docker command"
```

Task 12: Test clean-clone behavior and no-key behavior

Objective: Prove the submission works without relying on untracked local files.

Files:

- Create: `analysis/clean-clone-verification.md`
- No protected-file changes.

Create a temporary Git bundle or clone only inside the workspace, then verify:

```
git status --short
git bundle create analysis/clean-clone.bundle --all
git clone analysis/clean-clone.bundle analysis/clean-clone
cd analysis/clean-clone
docker compose build --no-cache coder
docker compose run --rm coder /in/notes.jsonl /out/predictions.jsonl
```

Before the run, make the test input available through the repository's documented path. Run once with no API keys:

```
env -u OPENAI_API_KEY -u ANTHROPIC_API_KEY docker compose run --rm coder
```

```
/in/notes.jsonl /out/predictions.jsonl
```

Expected:

- clean clone builds;
- no-key run exits 0;
- output has one line per input;
- no invalid codes;
- scorer completes;
- no hidden dependency on the developer's home directory.

Record exact commands, timings, line counts, and failures in `analysis/clean-clone-verification.md`. Remove only disposable clone artifacts after recording results; never remove the original archive or `public/`.

Commit:

```
git add analysis/clean-clone-verification.md
git commit -m "test: verify clean clone and no-key execution"
```

Task 13: Run structured development evaluation and error analysis

Objective: Measure the baseline honestly and identify targeted improvements without leakage.

Files:

- Create: `analysis/dev-evaluation.py`
- Create: `analysis/dev-evaluation.md`
- Create: `analysis/baseline-results/per-note.jsonl`
- Modify: `app/` only through new failing tests first.

Run the supplied scorer with per-note output if supported:

```
python public/score.py \
  --gold public/data/dev_notes.jsonl \
  --pred outputs/dev_predictions.jsonl \
  --mms public/data/mms-2026-01.json \
  --per-note analysis/baseline-results/per-note.jsonl
```

Analyze errors by style, chapter, leaf/residual status, and top-1 versus top-3. Do not add any rule without a failing regression test. For each improvement:

1. add a regression test reproducing the failure;
2. run it and verify failure;
3. make the smallest implementation change;

4. run the regression and full suite;
5. rerun the scorer;
6. record score delta and trade-off in `DECISIONS.md`;
7. commit.

Expected report fields:

```
input_notes: 60
prediction_lines: <integer>
missing_predictions: 0
invalid_codes: 0
top1: <measured>
top3: <measured>
score: <measured>
```

Task 14: Complete the gap analysis against the brief

Objective: Compare the actual implementation and verification evidence to every requirement in `public/BRIEF.md`.

Files:

- Create or modify: `analysis/gap-analysis.md`
- Create: `analysis/requirements-matrix.csv`

The matrix must contain columns:

```
requirement,source,implementation_evidence,status,test_command,result,remaining_r
```

Include rows for:

- exact Docker command;
- clean clone build;
- no API key;
- one output per input;
- valid MMS codes;
- top-three limit and order;
- incremental writes;
- 60-minute/600-note feasibility;
- API/proxy behavior if implemented;
- Git bundle with real history;
- one-page README;
- DECISIONS.md quality;
- hours-spent statement;

- generalisation uncertainty.

Run a read-only inspection:

```
python -m pytest -q
python public/score.py --gold public/data/dev_notes.jsonl --pred
outputs/dev_predictions.jsonl --mms public/data/mms-2026-01.json
git status --short
git log --oneline --decorate --all
```

Expected: every requirement is marked `PASS`, `PARTIAL`, or `OPEN` with concrete evidence; no requirement is marked pass based only on intention.

Commit:

```
git add analysis/gap-analysis.md analysis/requirements-matrix.csv
git commit -m "docs: complete brief-to-implementation gap analysis"
```

Task 15: Write the final README and decisions log

Objective: Produce the client-requested documentation without unsupported claims.

Files:

- Modify: `README.md` to one page or less when rendered.
- Modify: `DECISIONS.md` with actual entries from the implementation.

README must include:

- exact run command;
- architecture summary;
- no-key behavior;
- verified development score;
- honest hidden-score estimate or explicit statement that it is unknown;
- data added and provenance;
- evaluation beyond the 60 development notes, if any;
- one documented AI-tooling error that was caught, if one occurred;
- approximate hours spent.

`DECISIONS.md` entries must include date/time, commit, options, evidence, and revisit point. Do not pad the history with invented decisions. Use the actual number of decisions made.

Run:

```
python -m pytest -q
python public/score.py --gold public/data/dev_notes.jsonl --pred
outputs/dev_predictions.jsonl --mms public/data/mms-2026-01.json
```

Commit:

```
git add README.md DECISIONS.md
git commit -m "docs: finalize usage and engineering decisions"
```

Task 16: Generate the final Git bundle and submission packet

Objective: Create the final deliverables only after all tests and gap analysis pass.

Files:

- Create: `final-submission/<name>-Sugen-ICD11.bundle`
- Create: `final-submission/submission-checklist.md`
- Create: `final-submission/review-packet.md`
- Create: `final-submission/review-packet.pdf` after leaving plan mode and using the PDF workflow.

Run:

```
git status --short
git bundle create final-submission/RajivMehta-Sugen-ICD11.bundle --all
git bundle verify final-submission/RajivMehta-Sugen-ICD11.bundle
```

Expected: bundle verification reports a complete history and the intended branch/ref.

The checklist must verify:

- protected files still match their intake hashes;
- full test suite passes;
- exact Docker command passes from a clean clone;
- no-key path passes;
- scorer reports zero missing and invalid outputs;
- README is no more than one page;
- `DECISIONS.md` reflects real history;
- time statement is honest;
- bundle contains the final commits;
- final email recipient is `info@sugen.health`;
- no email is sent without explicit user approval.

Do not send the email or publicly post anything automatically.

---

4. Tests and validation summary

The minimum final validation sequence is:

```
cd /c/Users/rajiv/OneDrive/Desktop/work/sugen_health
python -m pytest -q
cp public/data/dev_notes.jsonl notes.jsonl
docker compose build --no-cache coder
docker compose run --rm coder /in/notes.jsonl /out/predictions.jsonl
python public/score.py --gold public/data/dev_notes.jsonl --pred
outputs/predictions.jsonl --mms public/data/mms-2026-01.json
python - <<'PY'
import json
from pathlib import Path
source = [json.loads(x) for x in Path('notes.jsonl').read_text().splitlines()
if x.strip()]
pred = [json.loads(x) for x in
Path('outputs/predictions.jsonl').read_text().splitlines() if x.strip()]
assert len(source) == len(pred), (len(source), len(pred))
assert [x['id'] for x in source] == [x['id'] for x in pred]
assert all(0 < len(x['codes']) <= 3 for x in pred)
print(f'contract PASS: {len(pred)} predictions')
PY
git bundle create final-submission/RajivMehta-Sugen-ICD11.bundle --all
git bundle verify final-submission/RajivMehta-Sugen-ICD11.bundle
```

Expected final evidence:

```
pytest: all tests passed
Docker build: exit 0
Docker run: exit 0
contract PASS: 60 predictions
score.py: 0 missing predictions, 0 invalid codes
bundle: complete history
```

The hidden 600-note score cannot be verified locally. Report it as unknown or as a clearly labelled estimate only.

---

5. Risks, trade-offs, and open questions

Risks

1. **Development-set leakage:** using gold labels in runtime would inflate the local score and damage trust. The submitted coder must load only the MMS catalog and explicitly documented static rules.
2. **Hidden generalisation distribution:** the 60 labelled notes do not represent all chapters, styles, or languages. A perfect dev score is not proof of hidden performance.
3. **Catalog titles are sparse:** title-only retrieval may miss synonyms and clinical phrasing. Add external data only after checking licensing, reproducibility, image size, and runtime constraints.
4. **API availability and cost:** optional APIs can fail through proxy, timeout, quota, or malformed output. The no-key path must remain complete and deterministic.
5. **Docker path mismatch:** changing the supplied relative layout can break the exact command. Keep the brief's required mounts and document any generated input/output path.
6. **Time window:** the exact deadline is unknown until the original email sent timestamp is available. Do not infer it from today's date.
7. **False confidence from local results:** only executed commands count as evidence.
8. **Audio transcription uncertainty:** the dad's short audio was transcribed with a local tiny model; the instruction is clear enough for planning, but ask for clarification if later feedback conflicts with the transcript.

Trade-offs

- **Standard library first:** fewer dependencies and more reliable Docker builds, at the cost of less sophisticated semantic matching.
- **Deterministic baseline before API:** easier to test and reproduce, but potentially lower hidden accuracy.
- **Optional API reranking:** may improve ambiguous notes, but adds cost, latency, proxy failure modes, and more code.
- **Small documented ruleset:** explainable and maintainable, but cannot cover every clinical synonym.
- **Frequent commits:** clearer history and easier review, but more commits to maintain.

Open questions for Dad's review

1. Should we permit any external medical synonym resource, or keep the solution entirely to supplied catalog plus code?
2. Should the optional API path be included, or should the submission stay deterministic and offline-first?
3. What exact email sent timestamp should be used to calculate the deadline?
4. Which hidden-score estimate, if any, is defensible after local validation?
5. Should final review artifacts include Markdown, DOCX, and PDF, with the PDF used for viewing/annotation and DOCX/Markdown for editing?
6. Is the current workspace layout acceptable, or should only the runtime files be kept at the root with analysis under `analysis/`?

---

6. Review packet and PDF handoff

This plan is the implementation plan. After plan approval and before coding begins, create a separate review packet under:

```
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\analysis\plan-review.md
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\analysis\plan-review.pdf
C:\Users\rajiv\OneDrive\Desktop\work\sugen_health\analysis\plan-review.docx
```

The packet should include:

- plain-language objective;
- folder map;
- requirements matrix;
- TDD workflow;
- no-leakage rule;
- Docker and scorer verification flow;
- risks and open questions;
- a KEEP / CHANGE / APPROVE feedback table for Dad.

The PDF must be generated and parsed/visually checked before being reported as complete. Do not create the PDF during plan mode; plan mode permits only this plan markdown file to be written.

---

7. Implementation stop conditions

Stop implementation and ask for review if:

- a protected company file would need modification;
- the exact Docker command cannot be made compatible without changing the brief's required paths;
- a proposed external data source has unclear licensing or reproducibility;
- the no-key path cannot produce a prediction for every input;
- a local score improvement requires development gold labels in runtime;
- the exact deadline becomes urgent but the email timestamp is missing;
- Dad's feedback conflicts with a client requirement.

The final submission is not ready until the full validation sequence, gap analysis, and bundle verification all pass with recorded evidence.